

Bias-Variance Tradeoff

In machine learning

Approximate a Target Function in Machine Learning

Supervised Machine learning approximating a **target function** (f) that maps input variables (X) to an output variable (Y).

$$Y = f(X)$$

Generalization is important because the **data we collect is only a sample**, it is **incomplete and noisy**.

Generalization refers to **how well** the concepts learned by a model apply to specific examples **not seen** by the model (**how well the model generalizes to new data**).

What is “Error” in Machine Learning?



Error is the difference in the **actual/ expected** output and the **predicted** output of the model.



It measures how well the model performs over a given set of data



To calculate error, loss function or cost function (Mean Squared Error (MSE)) is used

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y}_i)^2$$

y = actual or expected output of the model

$\bar{y}$ = predicted output of the model

n = Number of data points over which the error is calculated.

Errors

- Errors in any supervised learning algorithm comprises of 3 parts:
 - **Bias error**
 - we can attempt to minimize as much as possible - **reducible errors**
 - **Variance error**
 - we can attempt to minimize as much as possible - **reducible errors**
 - **Noise**
 - cannot eliminate - **irreducible error**

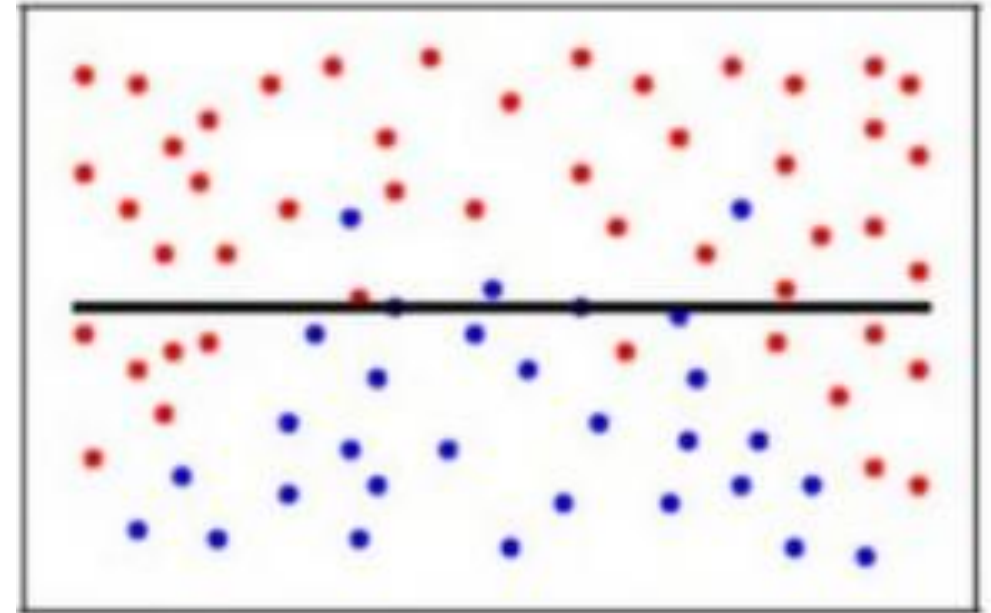
Total Error = Bias Error + Variance Error + Noise

What is bias?

- The model makes certain **assumptions** when it **trains** on the data provided.
- When it is introduced to the testing/validation data, these assumptions may not always be correct.
- When there is a **high bias error**, it results in a very simplistic model that does **not consider the variations very well (low variance)**.
- Since it does **not learn the training data very well**, it is called **Underfitting**.

$$y = f(x) + \epsilon$$

ϵ = error that is normally distributed



What is bias?

- Suppose the actual relationship between study hours and examination marks is curved:

$$y=5x^2+10$$

where:

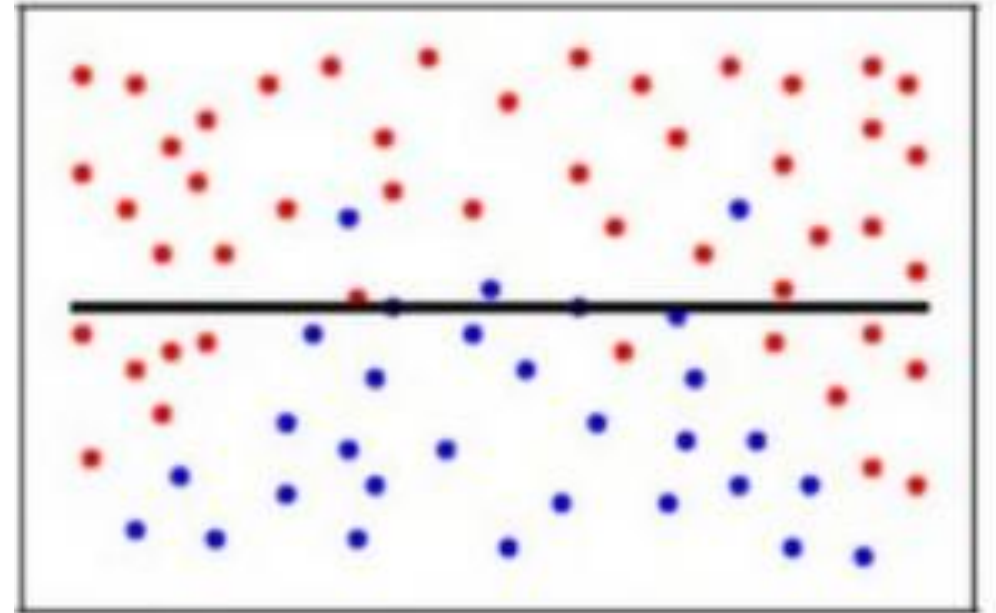
- x = study hours
- y = marks
- If we fit a straight-line model,

$$\hat{y} = b_0 + b_1x$$

The model may not represent the curved relationship properly. This produces high bias.

Characteristics of high bias

- The model is too **simple**.
- It performs **poorly** on training data.
- It also performs **poorly** on testing data.
- Training and testing **errors** are both **high**.
- It results in **underfitting**.

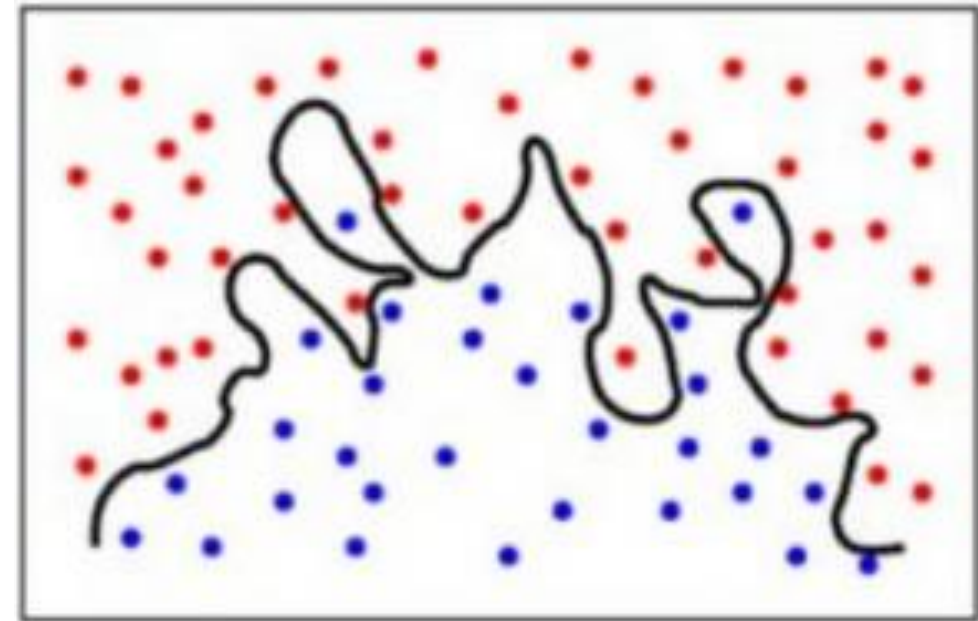


Models that may have high bias

- Linear regression applied to **nonlinear data**
- Logistic regression with **inadequate features**
- A decision tree with very **small depth**
- k -NN with a **very large** value of k
- Strongly **regularized** models

What is a Variance?

- A **high-variance** model **learns the training data too closely**, including noise and unimportant details.
- The model **learns too much** from the training data.
- When **testing** with new data, it is **unable to predict accurately** based on it.
- In the case of **high variance**, the model learns too much from the training data, it is called **overfitting**.



What is variance?

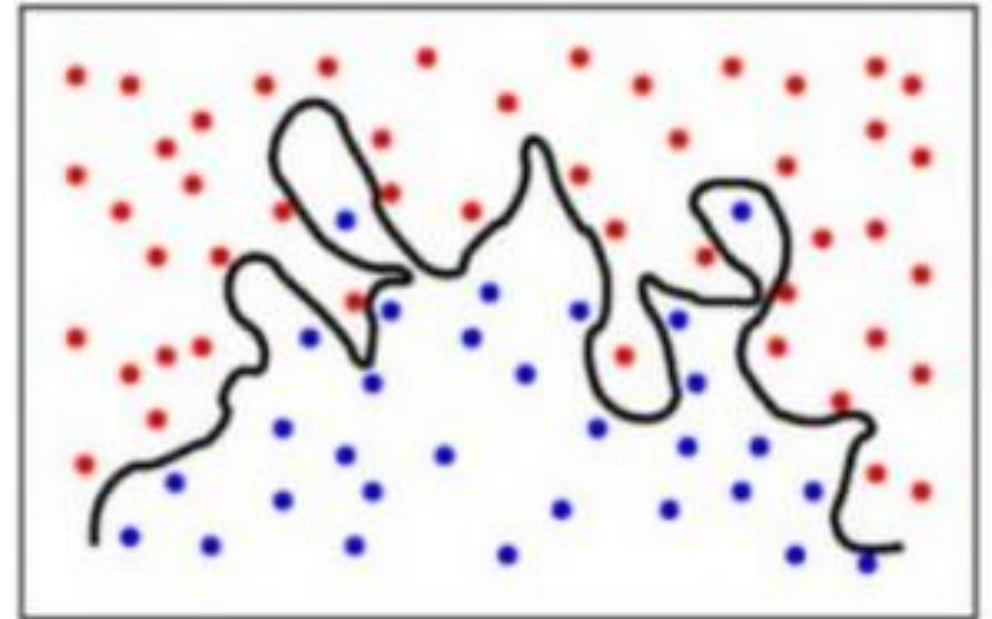
Example: Suppose we use a high-degree polynomial to predict marks from study hours:

$$\hat{y} = b_0 + b_1x + b_2x^2 + b_3x^3 + \dots + b_{10}x^{10}$$

- This **model may pass very close to every training observation.** However, it may produce incorrect predictions for new students.
- This produces high variance.

Characteristics of high variance

- The model is **too complex**.
- Training **error is very low**.
- Testing or **validation error is high**.
- There is a **large gap** between training and testing performance.
- It results in **overfitting**.

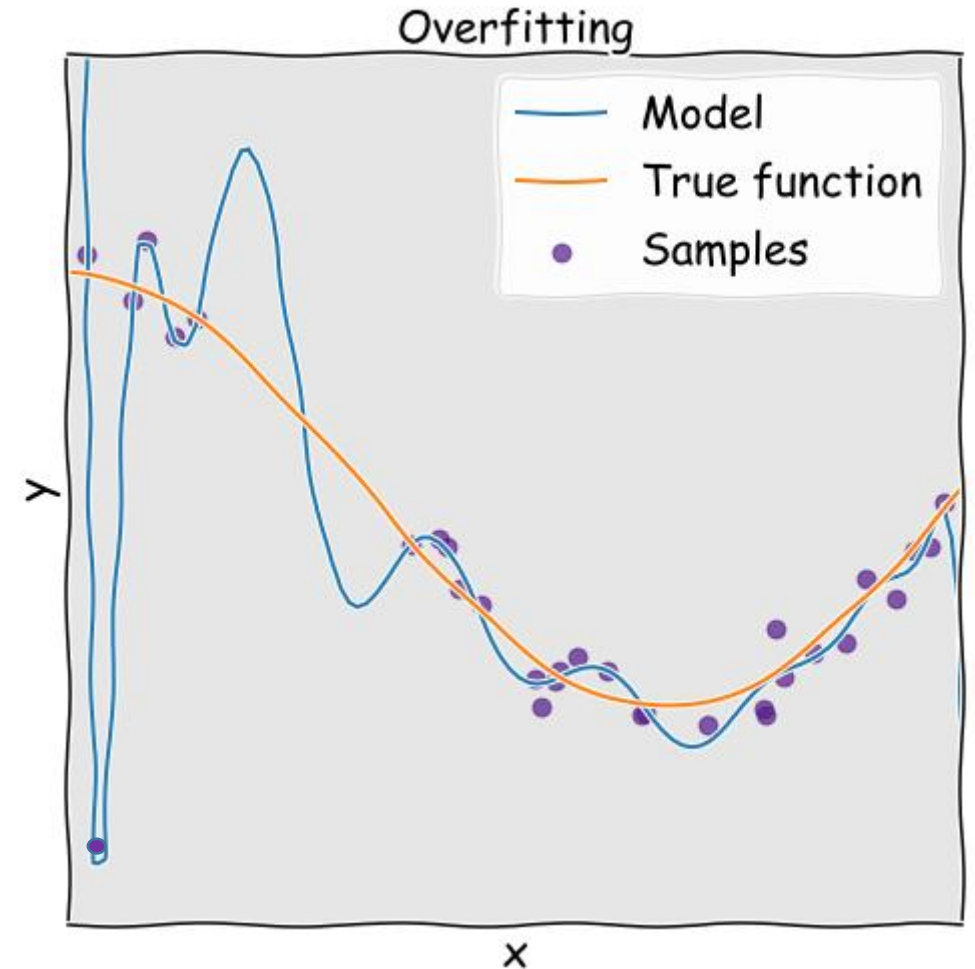


Models that may have high variance

- **Deep** decision trees
- High-degree **polynomial** regression
- k -NN with $k = 1$
- Neural networks with **many parameters** and **limited data**
- Models with very **weak regularization**

Overfitting

- When a model learns the **pattern and noise** in the data to such extent that it **hurts the performance of the model** on the new dataset, is termed **overfitting**.
- The model fits the data so well that it **interprets noise as patterns** in the data.
- Example: decision boundary is generated by non-linear models such as **decision trees**



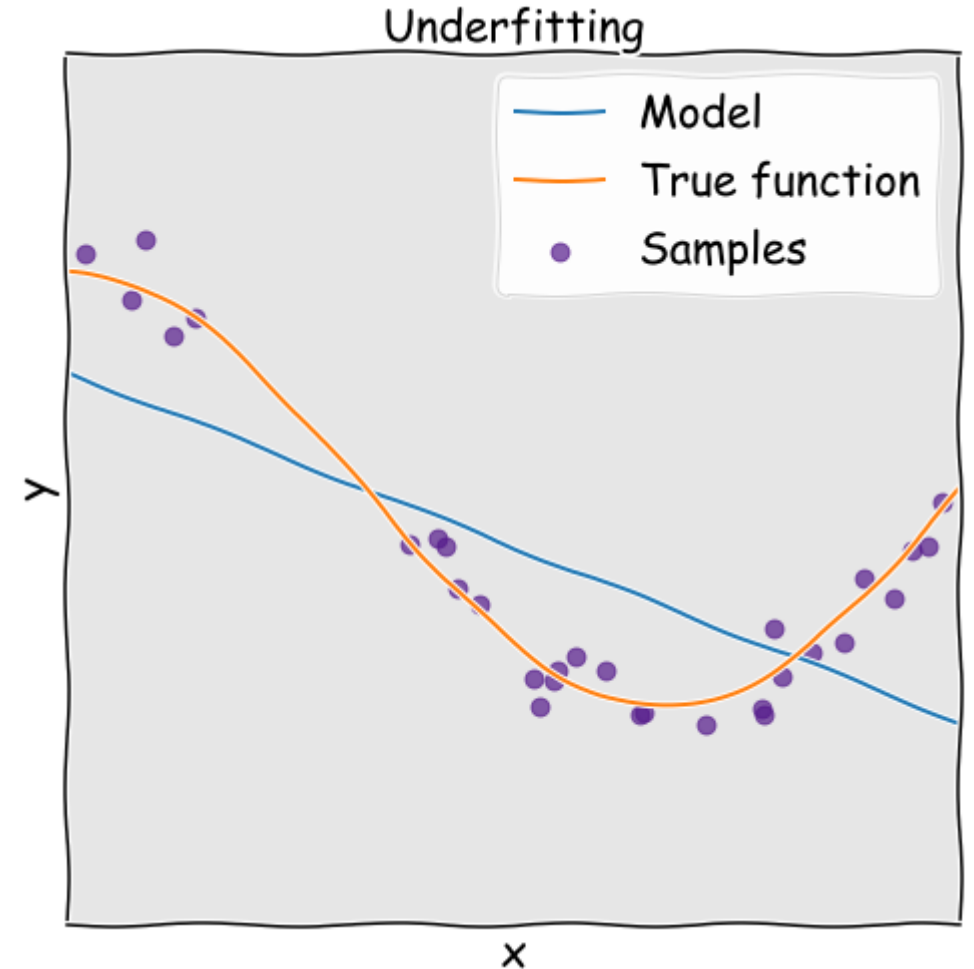
Overfitting - the model function has too much complexity (parameters) to fit the true function correctly

Overfitting

- Overfitting happens when the model learns the **training data too precisely**, including random noise.
- **Example:** fitting a degree-15 polynomial to **ten observations**.

Underfitting

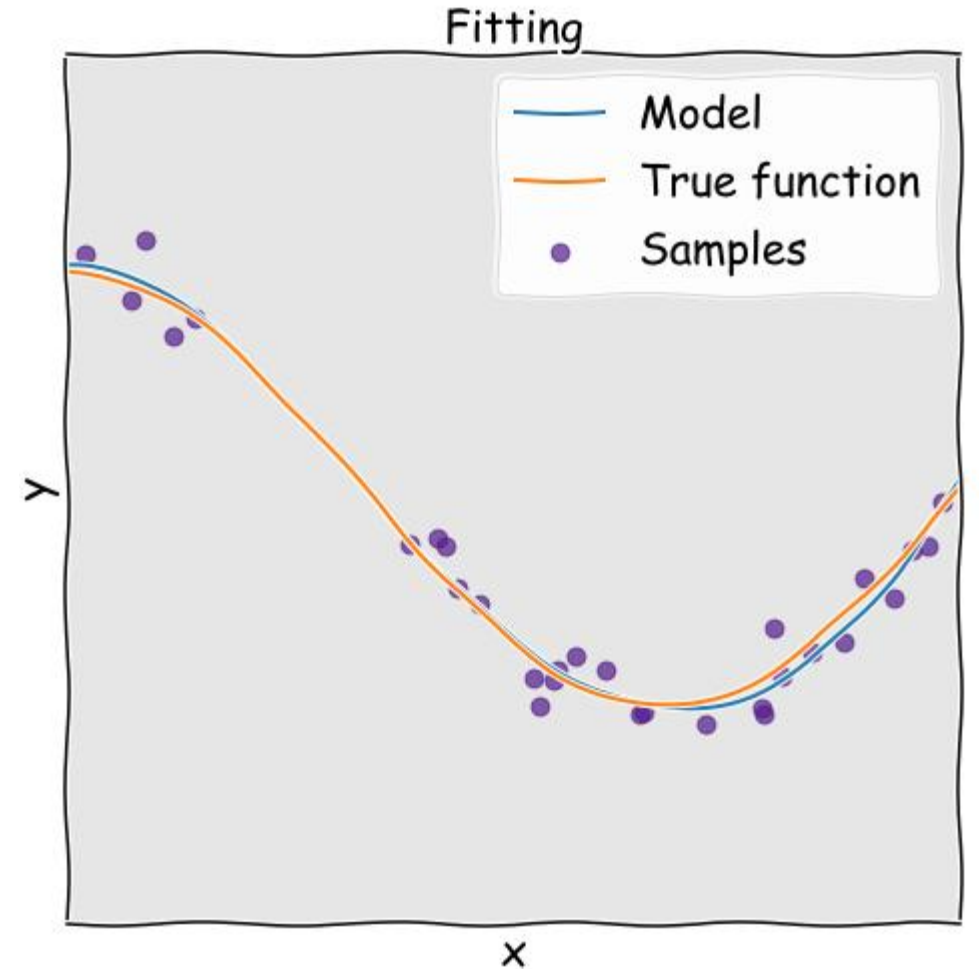
- When the model **neither learns from the training dataset** nor generalizes well on the **test dataset**, it is termed as underfitting.
- Underfitting happens when the model is **too simple** to learn the important patterns in the data.
- Example: fitting a straight line to strongly **curved data**.



Underfitting - the model function does not have enough complexity (parameters) to fit the true function correctly

Good Fit

- A good-fit model learns the important pattern but does not memorize the noise.
- *The spot in the middle of underfitting and overfitting is the good fit.*
- In the real world, getting a perfect fit model is very difficult.



An example of a well chosen model - the model function the right complexity (parameters) to fit the true function correctly.

Comparison

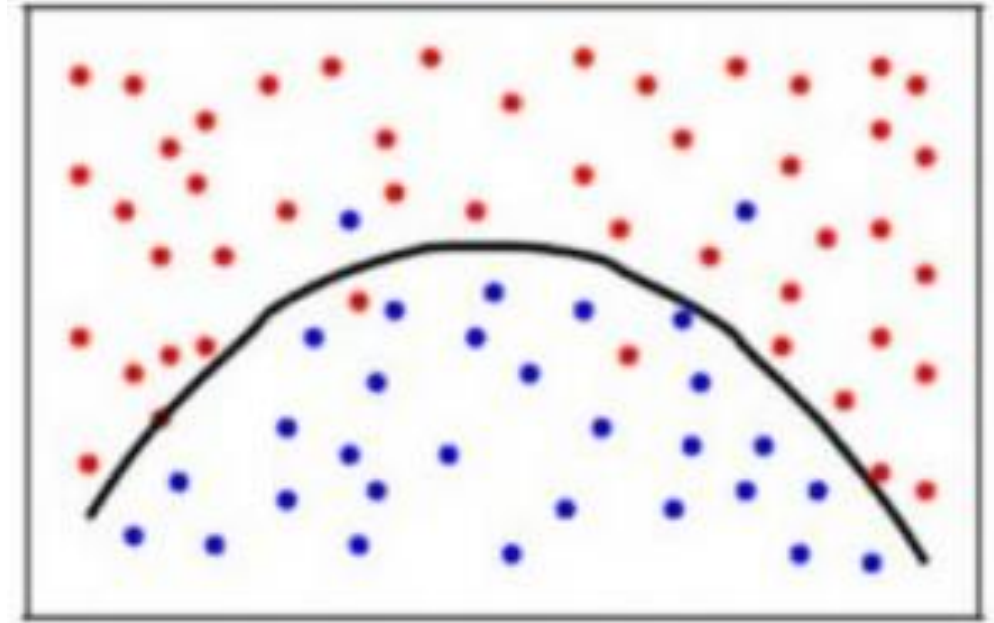
- Good fit
 - It performs **well on both** training and testing data.
- Overfitting:
 - **Good** performance on the **training data**, **poor** generalization to the **test data**.
- Underfitting:
 - **Poor** performance on the **training data** and **poor** generalization to **test data**.

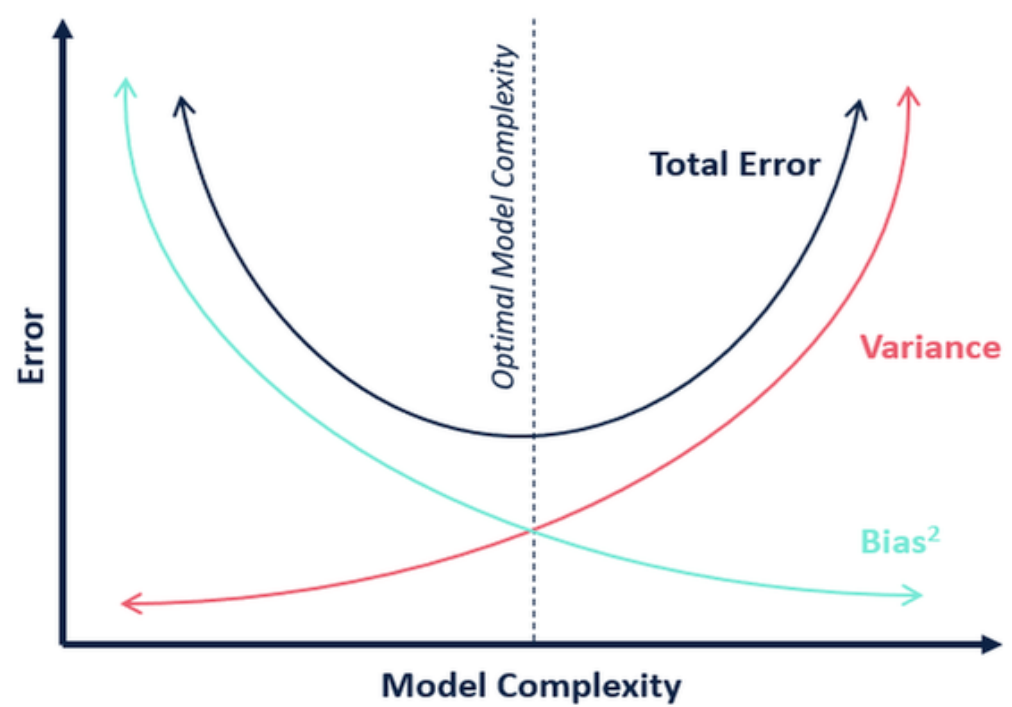
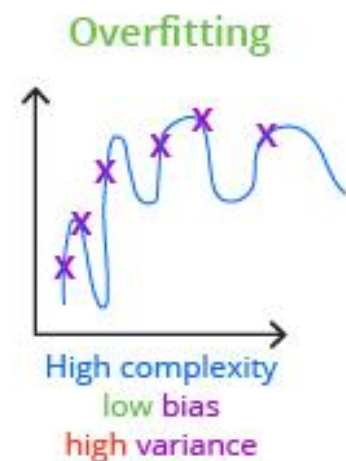
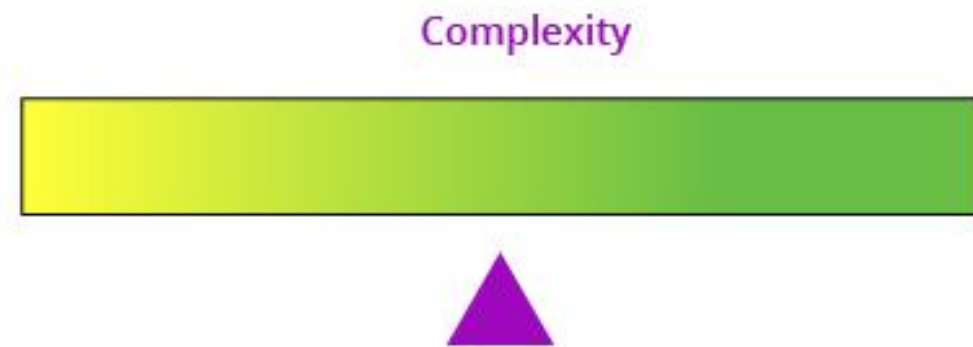
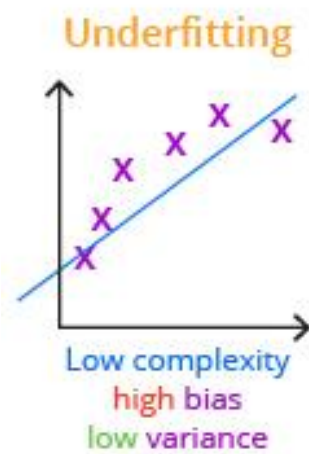
Underfitting and overfitting

Situation	Bias	Variance	Training error	Testing error
Underfitting	High	Low	High	High
Good fit	Moderate/low	Moderate/low	Low	Low
Overfitting	Low	High	Very low	High

Bias-Variance Tradeoff

- Though some points are classified incorrectly, the model generally fits most of the data points accurately.
- The balance between the Bias error and the Variance error is the **Bias-Variance Tradeoff**.

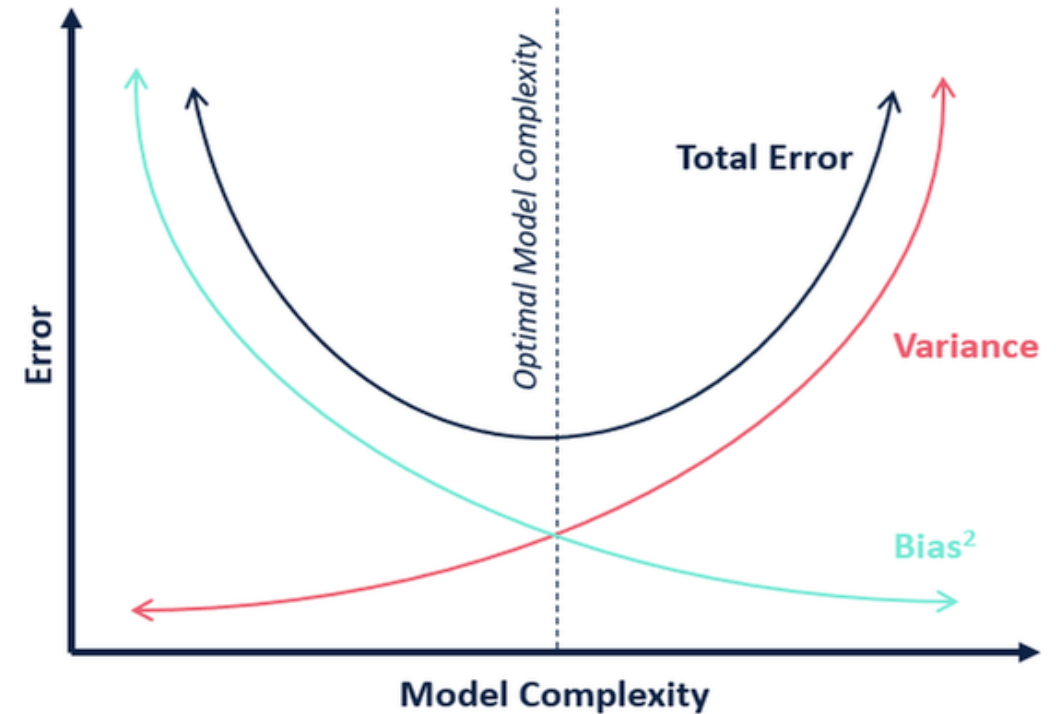


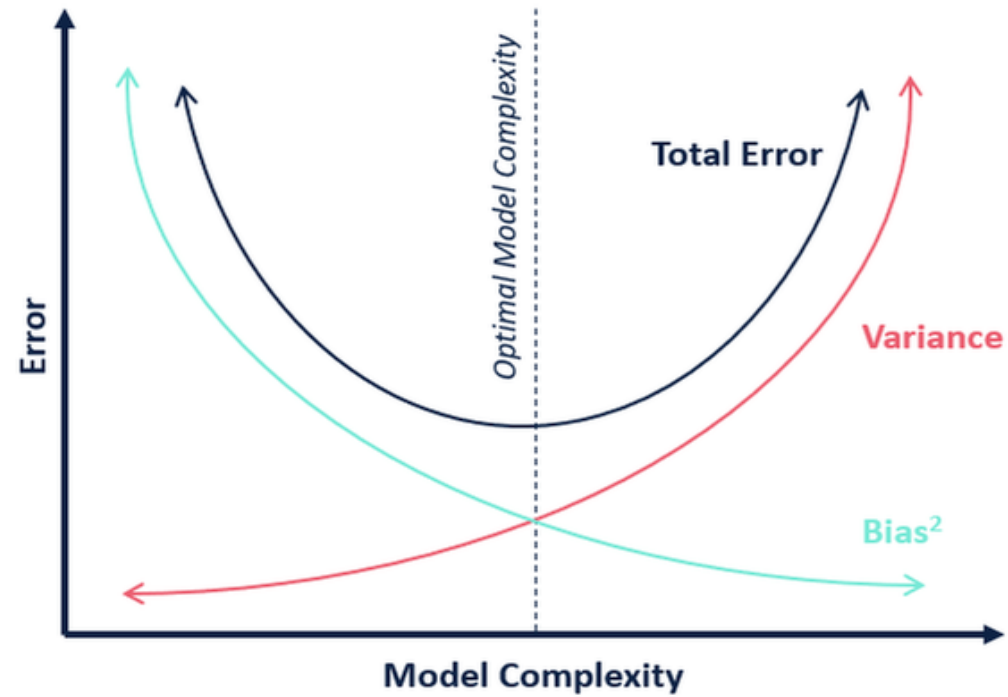


Bias–variance trade-off

Bias and variance are often affected in opposite directions.

- **Increasing** model complexity usually **decreases** bias.
- **Increasing** model complexity usually **increases** variance.
- **Decreasing** model complexity usually **increases** bias.
- **Decreasing** model complexity usually **decreases** variance.
- Therefore, we need to **select an appropriate level of complexity**.





Model complexity	Bias	Variance
Very low	High	Low
Moderate	Moderate/low	Moderate/low
Very high	Low	High

The best model is generally located between underfitting and overfitting.

Mathematical explanation

- For regression, the expected prediction error can be decomposed as:

$$\mathbb{E}(y - \hat{y})^2 = \text{Bias}(\hat{y})^2 + \text{variance}(\hat{y}) + \sigma^2$$

y is the actual output

$\hat{y}$ is the model prediction

$\text{Bias}(\hat{y})^2$ error caused by the assumptions

Variance is the error caused by sensitivity to the training data

σ^2 noise

- where:

$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$

Simple numerical example

- Suppose the true output for a particular input is: $y = f(x) = 50$.
- We train the same algorithm using five different training datasets.

- **Model A predictions:** 30, 31, 29, 30, 30

- Mean prediction:
$$\bar{\hat{y}} = \frac{30 + 31 + 29 + 30 + 30}{5} = 30$$

- Bias:
$$\text{Bias} = 30 - 50 = -20$$

- Squared bias:
$$\text{Bias}^2 = (-20)^2 = 400$$

- The predictions are **close to one another**, so **variance is low**. However, they are far from the true value (y).

Therefore, Model A has:

- High bias
- Low variance

- **Model B predictions:** 30, 70, 45, 65, 40

- Mean prediction:
$$\bar{\hat{y}} = \frac{30 + 70 + 45 + 65 + 40}{5} = 50$$

- The mean prediction **equals** the **true value (y)**, so the bias is zero.
However, the predictions are **widely scattered**.

Therefore, Model B has:

- Low bias
- High variance

Calculation for Model B

- Predictions: 30, 70, 45, 65, 40
- Mean:

$$\bar{\hat{y}} = 50$$

Variance:

$$\begin{aligned}\text{Variance} &= \frac{(30 - 50)^2 + (70 - 50)^2 + (45 - 50)^2 + (65 - 50)^2 + (40 - 50)^2}{5} \\ &= \frac{400 + 400 + 25 + 225 + 100}{5} \\ &= \frac{1150}{5} \\ &= 230\end{aligned}$$

Thus, Model B has zero bias but high variance.

- **Model C predictions:** 48, 51, 49, 52, 50
- Mean prediction: $\bar{\hat{y}} = 50$
- The predictions are close to the true value and close to one another.

Therefore, Model C has:

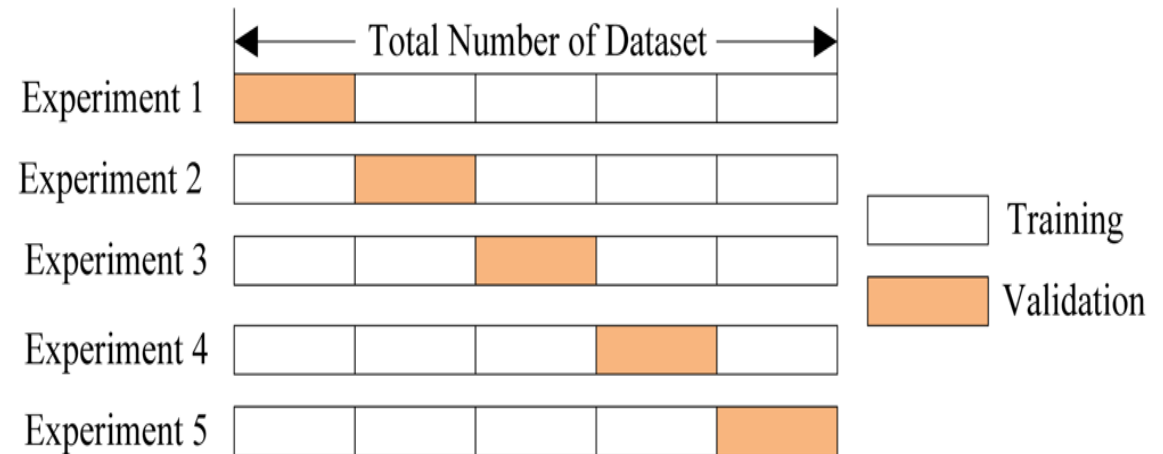
- Low bias
- Low variance

How to tackle overfitting

- Prevent the model from overfitting by using **hyperparameter tuning**.
- **K-fold cross-validation** to do hyperparameter tuning.
- In K-fold cross-validation
 - Divide the dataset into three parts.
 - Training part
 - Validation part
 - Test part

How k-fold cross-validation works ?

- Example: **5-fold cross-validation**
 - Training data set will be divided into 5 parts (randomly 4 parts will be selected for **training** and 1 part for **validation**).
- This will be repeated 5 times, and the average of the metric will be the final metric for that epoch.



Five-Fold Cross-Validation

How k-fold cross-validation works ?

In one epoch, the depth of the tree and split value - **number of data points** at each node after which the node will be split is fixed.

Example:

- In an epoch, we can have the depth of the tree to be 100 and split value at 200.
- Let the training dataset D is divided into D1, D2, D3, D4, and D5.

Tree Depth	N-Split Value	Training Dataset	Validation Dataset	Metric Value	Final Metric Value
100	200	D1, D2, D3, D4	D5	a5	$A1 = (a1+a2+a3+a4+a5)/5$
100	200	D1, D2, D3, D5	D4	a4	
100	200	D1, D2, D4, D5	D3	a3	
100	200	D1, D3, D4, D5	D2	a2	
100	200	D2, D3, D4, D5	D1	a1	

- Let us take accuracy in each epoch as a1, a2, a3, a4, and a5.
- After training A1 is the average training accuracy.

How k-fold cross-validation works ?

- If both the **training accuracy and test accuracy are close**, then the model has **not overfit**.
- If the **training accuracy is very good** and the **test** accuracy is **poor**, then the model has **overfitted**.
- If the **training** accuracy and **test** accuracy - **low**, then the model has **underfit**.
- If the model is underfitting or overfitting, then we have to **change the value of the hyperparameter** and again retrain the model until we get a good fit.

Hyperparameter Tuning

- In hyperparameter tuning, we **take a range of all the hyperparameter** and then we **monitor the cross-validation accuracy** on all the possible combinations of hyperparameters.
- We take that **hyperparameter** that gave the **best accuracy** (here we have taken accuracy as a metric).
- Then we train the model with that **hyperparameter** and then test it.
- Following is how cross-validation accuracy is calculated for each **hyperparameter** combination.

Epoch	Tree Depth	N-Split Value	Training Dataset	Validation Dataset	Metric Value	Final Metric Value
1	100	200	D1, D2, D3, D4	D5	a5	$A1 = (a1+a2+a3+a4+a5)/5$
	100	200	D1, D2, D3, D5	D4	a4	
	100	200	D1, D2, D4, D5	D3	a3	
	100	200	D1, D3, D4, D5	D2	a2	
	100	200	D2, D3, D4, D5	D1	a1	
2	500	200	D1, D2, D3, D4	D5	a5	$A2 = (a1+a2+a3+a4+a5)/5$
	500	200	D1, D2, D3, D5	D4	a4	
	500	200	D1, D2, D3, D5	D3	a3	
	500	200	D1, D3, D4, D5	D2	a2	
	500	200	D2, D3, D4, D5	D1	a1	

Decision trees can easily overfit the training data, especially if they are allowed to grow too deep. To prevent overfitting, you can use parameters like **max_depth**, **min_samples_split**, and **min_samples_leaf** to control the size of the tree.

Avoid Overfitting - Reduce Number of features

- Plotting hypothesis:
 - For **lower dimensional datasets**: it is possible to **plot** the hypothesis to check if is overfit or not.
 - When the **dimensionality of the dataset increases** beyond the limit of visualization: same strategy **cannot be applied**.
 - So, **plotting hypothesis** may **not always work**.
- Need other methods to address overfitting.
- There are two options to avoid the overfitting:
 - **Reduce the number of features**
 - **Regularization**

Reduce the number of features

- Mostly overfitting is observed when the **number of features is high**, and the number of **training examples are less**.
- In such cases **reducing the number of features** can help avoid the issue of overfitting.
- Reducing the number of features can be done manually or using **model selection algorithms** which help decide **which features to eliminate**.
- Disadvantage of this approach: **removing features is sometimes equivalent to removing information**.

Avoid overfitting - Regularization

- Regularization is another approach to avoid overfitting.
- In regularization - keep all the features but reduce magnitude/values of parameters β_j .
- This technique works well, when there exists a lot features and each contributes a bit to the prediction,
- Regularization works well **when there are a lot of slightly useful features.**

Regularization

- The main ideas in regularization:
 - maintain small values for the parameters $\beta_0, \beta_1, \dots, \beta_n$
 - This keeps the hypothesis simple and less prone to overfitting.
- Mathematically, regularization is achieved by modifying the cost function $J(\beta)$ as follows,

$$\min \left[\sum (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right]$$

λ is a regularization parameter

If value of β_j is increased, it would consequently increase the cost

λ controls the trade off between the goal of fitting the training set well and the goal of keeping the parameters small and the hypothesis simple.

if the value of λ is kept very large, it would fail to fit the dataset and would give a underfit hypothesis.

Thank you